

```
pip install PyPDF2
```

Collecting PyPDF2

Downloading pypdf2-3.0.1-py3-none-any.whl.metadata (6.8 kB)
 Downloading pypdf2-3.0.1-py3-none-any.whl (232 kB)
 232.6/232.6 kB 5.2 MB/s eta 0:00:00

Installing collected packages: PyPDF2
 Successfully installed PyPDF2-3.0.1

```
pip install python-docx
```

Collecting python-docx

Downloading python_docx-1.2.0-py3-none-any.whl.metadata (2.0 kB)
 Requirement already satisfied: lxml>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (6.0.2)
 Requirement already satisfied: typing_extensions>=4.9.0 in /usr/local/lib/python3.12/dist-packages (from python-docx)
 Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)
 253.0/253.0 kB 7.9 MB/s eta 0:00:00

Installing collected packages: python-docx
 Successfully installed python-docx-1.2.0

```
from google.colab import files
uploaded = files.upload()
if uploaded:
    print("Success")
else:
    print("Failed")
```

index.jpg
index.jpg(image/jpeg) - 4652 bytes, last modified: 3/29/2026 - 100% done
 Saving index.jpg to index.jpg
 Success

```
# Import modules
import PyPDF2

# Open the uploaded PDF from /content
pdfFileObj = open('/content/sample.pdf', 'rb')

# Create PDF reader object (updated syntax)
pdfReader = PyPDF2.PdfReader(pdfFileObj)

# Print number of pages
print(len(pdfReader.pages))

# Get first page
pageObj = pdfReader.pages[0]

# Extract text
print(pageObj.extract_text())

# Close file
pdfFileObj.close()
```

```
1
Welcome to Smallpdf
Digital Documents--All In One Place
Access Files Anytime, Anywhere Enhance Documents in One Click
Collaborate With Others With the new Smallpdf experience, you can
freely upload, organize, and share digital
documents. When you enable the 'Storage'
option, we'll also store all processed files here.
You can access files stored on Smallpdf from
your computer, phone, or tablet. We'll also
sync files from the Smallpdf Mobile App to our
online portalWhen you right-click on a file, we'll present
you with an array of options to convert,
compress, or modify it.
Forget mundane administrative tasks. With
Smallpdf, you can request e-signatures, send
large files, or even enable the Smallpdf G Suite
App for your entire organization. Ready to take document management to the next level?
```

```
# import docx NOT python-docx
import docx

# create an instance of a word document
doc = docx.Document()

# add a heading of level 0 (largest heading)
doc.add_heading('Heading for the document', 0)

# add a paragraph and store
# the object in a variable
doc_para = doc.add_paragraph('Your paragraph goes here, ')

# add a run i.e, style like
# bold, italic, underline, etc.
doc_para.add_run('hey there, bold here').bold = True
doc_para.add_run(', and ')
doc_para.add_run('these words are italic').italic = True

# add a page break to start a new page
doc.add_page_break()

# add a heading of level 2
doc.add_heading('Heading level 2', 2)

# pictures can also be added to our word document
# width is optional
doc.add_picture("/content/index.jpg")

# now save the document to a location
doc.save('new_doc')
```

```
pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

#Sentence Tokenization

```
sentence_data = "The First sentence is about Python. The Second: about Django. You can learn Python,Django and Data
nltk_tokens = nltk.sent_tokenize(sentence_data)
print (nltk_tokens)
```

```
['The First sentence is about Python.', 'The Second: about Django.', 'You can learn Python,Django and Data Ananlysis']
```

#Non English language Tokenization

```
german_tokenizer = nltk.data.load('tokenizers/punkt/german.pickle')
german_tokens=german_tokenizer.tokenize('Wie geht es Ihnen? Gut, danke.')
print(german_tokens)
```

```
['Wie geht es Ihnen?', 'Gut, danke.']
```

#Word Tokenization

```
word_data = "It originated from the idea that there are readers who prefer learning new skills from the comforts of
nlk_tokens = nltk.word_tokenize(word_data)
print (nlk_tokens)
```

```
['It', 'originated', 'from', 'the', 'idea', 'that', 'there', 'are', 'readers', 'who', 'prefer', 'learning', 'new', 's
```

#Word Tokenization

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
```

```
#Dummy text
txt = "He is a boy. "\
      "She is a girl"
```

```
word_tokens = word_tokenize(txt)
```

```
print(word_tokens)
```

```
['He', 'is', 'a', 'boy', '.', 'She', 'is', 'a', 'girl']
```

#Part of Speech (POS) tagging

```
import nltk
nltk.download('averaged_perceptron_tagger')
from nltk.tokenize import word_tokenize
text = word_tokenize("Hello welcome to the world of to learn Categorizing and POS Tagging with NLTK and Python")
nltk.pos_tag(text)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[('Hello', 'NNP'),
 ('welcome', 'NN'),
 ('to', 'TO'),
 ('the', 'DT'),
 ('world', 'NN'),
 ('of', 'IN'),
 ('to', 'TO'),
 ('learn', 'VB'),
 ('Categorizing', 'NNP'),
 ('and', 'CC'),
 ('POS', 'NNP'),
 ('Tagging', 'NNP'),
 ('with', 'IN'),
 ('NLTK', 'NNP'),
 ('and', 'CC'),
 ('Python', 'NNP')]
```

```
import nltk
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
True
```

```
from nltk.corpus import stopwords
print(stopwords.words('english'))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't",
```

#Stopwords removal from sentence

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

```
example_sent = """This is a sample sentence,
                  showing off the stop words filtration."""
```

```
stop_words = set(stopwords.words('english'))
```

```
word_tokens = word_tokenize(example_sent)

filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]

filtered_sentence = []

for w in word_tokens:
    if w not in stop_words:
        filtered_sentence.append(w)

print("Tokenized:", word_tokens)
print("Stop Words Removed:", filtered_sentence)
```

Tokenized: ['This', 'is', 'a', 'sample', 'sentence', ',', 'showing', 'off', 'the', 'stop', 'words', 'filtration', '.']
 Stop Words Removed: ['This', 'sample', 'sentence', ',', 'showing', 'stop', 'words', 'filtration', '.']

```
from google.colab import files
uploaded = files.upload()
if uploaded:
    print("Success")
else:
    print("Failed")
```

Choose Files | text.txt
 text.txt(text/plain) - 27 bytes, last modified: 3/29/2026 - 100% done
 Saving text.txt to text.txt
 Success

```
#Stopwords from input file

import io
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# word_tokenize accepts
# a string as an input, not a file.
stop_words = set(stopwords.words('english'))
file1 = open("/content/text.txt")

# Use this to read file content as a stream:
line = file1.read()
words = line.split()
for r in words:
    if not r in stop_words:
        appendFile = open('filteredtext.txt','a')
        appendFile.write(" "+r)
        appendFile.close()
```

```
#Stemming

import nltk
from nltk.stem.porter import PorterStemmer
porter_stemmer = PorterStemmer()

word_data = "It vijaying meeting better vijayed vijays eats skills originated from the idea that there are readers w
# First Word tokenization
nltk_tokens = nltk.word_tokenize(word_data)
#Next find the roots of the word
for w in nltk_tokens:
    print("Actual: %s Stem: %s" % (w,porter_stemmer.stem(w)))
```

```
Actual: It Stem: it
Actual: vijaying Stem: vijay
Actual: meeting Stem: meet
Actual: better Stem: better
Actual: vijayed Stem: vijay
Actual: vijays Stem: vijay
Actual: eats Stem: eat
Actual: skills Stem: skill
Actual: originated Stem: origin
Actual: from Stem: from
Actual: the Stem: the
Actual: idea Stem: idea
Actual: that Stem: that
```

```

Actual: there Stem: there
Actual: are Stem: are
Actual: readers Stem: reader
Actual: who Stem: who
Actual: prefer Stem: prefer
Actual: learning Stem: learn
Actual: new Stem: new
Actual: skills Stem: skill
Actual: from Stem: from
Actual: the Stem: the
Actual: comforts Stem: comfort
Actual: of Stem: of
Actual: their Stem: their
Actual: drawing Stem: draw
Actual: rooms Stem: room

```

#Lemmatization

```

import nltk
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()

word_data = "It studies densely is better meeting studying vijaying vijayed vijays skills originated from the idea
nltk_tokens = nltk.word_tokenize(word_data)
for w in nltk_tokens:
    print("Actual: %s Lemma: %s" % (w, wordnet_lemmatizer.lemmatize(w)))

```

[nltk_data] Downloading package wordnet to /root/nltk_data...

```

Actual: It Lemma: It
Actual: studies Lemma: study
Actual: densely Lemma: densely
Actual: is Lemma: is
Actual: better Lemma: better
Actual: meeting Lemma: meeting
Actual: studying Lemma: studying
Actual: vijaying Lemma: vijaying
Actual: vijayed Lemma: vijayed
Actual: vijays Lemma: vijays
Actual: skills Lemma: skill
Actual: originated Lemma: originated
Actual: from Lemma: from
Actual: the Lemma: the
Actual: idea Lemma: idea
Actual: that Lemma: that
Actual: there Lemma: there
Actual: are Lemma: are
Actual: readers Lemma: reader
Actual: who Lemma: who
Actual: prefer Lemma: prefer
Actual: learning Lemma: learning
Actual: new Lemma: new
Actual: skills Lemma: skill
Actual: from Lemma: from
Actual: the Lemma: the
Actual: comforts Lemma: comfort
Actual: of Lemma: of
Actual: their Lemma: their
Actual: drawing Lemma: drawing
Actual: rooms Lemma: room

```

#Expt.No.7 2nd Operation

```

import pandas as pd
import sklearn as sk
import math

```

```

first_sentence = "Data Science is the best job of the 21st century"
second_sentence = "Machine learning is the key for data science"

```

```

#split so each word have their own string
first_sentence = first_sentence.split(" ")
second_sentence = second_sentence.split(" ")#join them to remove common duplicate words
total= set(first_sentence).union(set(second_sentence))
print(total)

```

```
{'the', 'for', 'Data', '21st', 'key', 'Machine', 'best', 'is', 'science', 'of', 'century', 'data', 'job', 'Science',
```

```
#count the words

wordDictA = dict.fromkeys(total, 0)
wordDictB = dict.fromkeys(total, 0)
for word in first_sentence:
    wordDictA[word]+=1

for word in second_sentence:
    wordDictB[word]+=1

pd.DataFrame([wordDictA, wordDictB])
```

	the	for	Data	21st	key	Machine	best	is	science	of	century	data	job	Science	learning
0	2	0	1	1	0	0	1	1	0	1	1	0	1	1	0
1	1	1	0	0	1	1	0	1	1	0	0	1	0	0	1

```
#Compute Term Frequency(TF)

def computeTF(wordDict, doc):
    tfDict = {}
    corpusCount = len(doc)
    for word, count in wordDict.items():
        tfDict[word] = count/float(corpusCount)
    return(tfDict)

#running our sentences through the tf function:
tfFirst = computeTF(wordDictA, first_sentence)
tfSecond = computeTF(wordDictB, second_sentence)

#Converting to dataframe for visualization
pd.DataFrame([tfFirst, tfSecond])
```

	the	for	Data	21st	key	Machine	best	is	science	of	century	data	job	Science	learning
0	0.200	0.000	0.1	0.1	0.000	0.000	0.1	0.100	0.000	0.1	0.1	0.000	0.1	0.1	0.000
1	0.125	0.125	0.0	0.0	0.125	0.125	0.0	0.125	0.125	0.0	0.0	0.125	0.0	0.0	0.125

```
#Compute Inverse Document Frequency(IDF)

def computeIDF(docList):
    idfDict = {}
    N = len(docList)

    idfDict = dict.fromkeys(docList[0].keys(), 0)
    for word, val in idfDict.items():
        idfDict[word] = math.log10(N / (float(val) + 1))

    return(idfDict)

#inputing our sentences in the log file
idfs = computeIDF([wordDictA, wordDictB])
```

```
#Compute Term Frequency(TF) - Inverse Document Frequency(IDF)

def computeTFIDF(tfBow, idfs):
    tfidf = {}
    for word, val in tfBow.items():
        tfidf[word] = val*idfs[word]
    return(tfidf)

#running our two sentences through the IDF:
idfFirst = computeTFIDF(tfFirst, idfs)
idfSecond = computeTFIDF(tfSecond, idfs)

#putting it in a dataframe
pd.DataFrame([idfFirst, idfSecond])
```

	the	for	Data	21st	key	Machine	best	is	science	of	century	data	job	Science	learning
0	0.060206	0.000000	0.030103	0.030103	0.000000	0.000000	0.030103	0.030103	0.000000	0.030103	0.030103	0.000000	0.030103	0.030103	0.000000
1	0.037629	0.037629	0.000000	0.000000	0.037629	0.037629	0.000000	0.037629	0.037629	0.000000	0.000000	0.037629	0.000000	0.000000	0.037629

```
#Compute TF-IDF

#first step is to import the library
from sklearn.feature_extraction.text import TfidfVectorizer

#for the sentence, make sure all words are lowercase or you will run #into error. for simplicity, I just made the sam
firstV= "Data Science is the sexiest job of the 21st century"
secondV= "machine learning is the key for data science"

#calling the TfidfVectorizer
vectorize= TfidfVectorizer()

#fitting the model and passing our sentences right away:
response= vectorize.fit_transform([firstV, secondV])

print(response)
```

<Compressed Sparse Row sparse matrix of dtype 'float64'
with 17 stored elements and shape (2, 13)>

Coords	Values
(0, 2)	0.24342026926924518
(0, 10)	0.24342026926924518
(0, 4)	0.24342026926924518
(0, 12)	0.48684053853849035
(0, 11)	0.34211869506421816
(0, 5)	0.34211869506421816
(0, 9)	0.34211869506421816
(0, 0)	0.34211869506421816
(0, 1)	0.34211869506421816
(1, 2)	0.28986933576883284
(1, 10)	0.28986933576883284
(1, 4)	0.28986933576883284
(1, 12)	0.28986933576883284
(1, 8)	0.40740123733358447
(1, 7)	0.40740123733358447
(1, 6)	0.40740123733358447
(1, 3)	0.40740123733358447